

WEEK 11

Task 1

Skill objective:

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

Implementation / work:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

typedef enum { RUNNING, STOPPED, DONE } Status;

typedef struct {
    int jid;
    pid_t pid;
    Status status;
    char cmd[48];
} JobRecord;

void list_jobs(JobRecord jobs[], int n) {
    printf("JID      PID      STATUS      COMMAND\n");
    printf("-----\n");
    for (int i = 0; i < n; i++) {
        const char *stat_str = (jobs[i].status == RUNNING) ? "Running" :
                               (jobs[i].status == STOPPED) ? "Stopped" : "Done";
        printf("[%d]   %-7d  %-10s  %s\n",
               jobs[i].jid, jobs[i].pid, stat_str, jobs[i].cmd);
    }
}

int main() {
    JobRecord active_jobs[] = {
        {1, 2301, RUNNING, "sleep 100 &"},
        {2, 2305, STOPPED, "vim server.conf"},
        {3, 2312, RUNNING, "find / -name '*.log' > logs.txt &"},
        {4, 2318, DONE, "tar -czf backup.tar.gz src/"},
    };
    printf("[Task 1] Listing active shell jobs and formatted statuses:\n\n");
    list_jobs(active_jobs, 4);
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week11_task1.c
teja27@TEJA:~$ gcc week11_task1.c -o week11_task1
teja27@TEJA:~$ ./week11_task1
[Task 1] Listing active shell jobs and formatted statuses:

JID      PID      STATUS      COMMAND
-----
[1]   2301     Running     sleep 100 &
[2]   2305     Stopped      vim server.conf
[3]   2312     Running     find / -name '*.log' > logs.txt &
[4]   2318     Done         tar -czf backup.tar.gz src/
teja27@TEJA:~$
```

Task 2

Skill objective:

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("[Task 2] Foreground Switching Simulation (fg %1)\n");
    pid_t pid = fork();
    if (pid == 0) {
        /* Child simulated process */
        usleep(400000);
        printf("[Job 1] Child process (PID %d) resuming foreground execution...\n", getpid());
        usleep(300000);
        _exit(0);
    }

    printf("1. Target Job identified: JID [1] -> PID: %d\n", pid);
    printf("2. Sending SIGCONT signal to resume stopped/background job...\n");
    kill(pid, SIGCONT);

    printf("3. Transferring terminal control to PID %d (tcsetpgrp simulation)\n", pid);
    int status;
    waitpid(pid, &status, WUNTRACED);

    printf("4. Process completed. Terminal ownership restored to shell.\n");
    printf("5. Job state updated in Job Table: JID [1] -> TERMINATED.\n");
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week11_task2.c
teja27@TEJA:~$ gcc week11_task2.c -o week11_task2
teja27@TEJA:~$ ./week11_task2
[Task 2] Foreground Switching Simulation (fg %1)
1. Target Job identified: JID [1] -> PID: 864
2. Sending SIGCONT signal to resume stopped/background job...
3. Transferring terminal control to PID 864 (tcsetpgrp simulation)
4. Process completed. Terminal ownership restored to shell.
5. Job state updated in Job Table: JID [1] -> TERMINATED.
teja27@TEJA:~$
```

Task 3

Skill objective:

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void analyze_signal_mask() {
    sigset_t old_mask, block_mask;
    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGINT); /* Ctrl+C */
    sigaddset(&block_mask, SIGTSTP); /* Ctrl+Z */

    printf("[Task 3] Blocking SIGINT and SIGTSTP via sigprocmask(SIG_BLOCK)...\n");
    sigprocmask(SIG_BLOCK, &block_mask, &old_mask);

    printf("Critical section active: Shell data structures safely protected.\n");
    printf("Process Group ID: %d | Foreground Group: %d\n", getpgrp(), tcgetpgrp(STDIN_FILENO));

    printf("Restoring signal mask via sigprocmask(SIG_SETMASK)...\n");
    sigprocmask(SIG_SETMASK, &old_mask, NULL);
    printf("Signal delivery unblocked successfully.\n");
}

int main() {
    analyze_signal_mask();
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week11_task3.c
teja27@TEJA:~$ gcc week11_task3.c -o week11_task3
teja27@TEJA:~$ ./week11_task3
[Task 3] Blocking SIGINT and SIGTSTP via sigprocmask(SIG_BLOCK)...
Critical section active: Shell data structures safely protected.
Process Group ID: 878 | Foreground Group: -1
Restoring signal mask via sigprocmask(SIG_SETMASK)...
Signal delivery unblocked successfully.
teja27@TEJA:~$ █
```

Task 4

Skill objective:

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("[Task 4] Resume Stopped Jobs in Background (bg %1)\n");
    pid_t pid = fork();
    if (pid == 0) {
        usleep(3000000);
        printf("[Background Worker PID %d] Processing asynchronous tasks...\n", getpid());
        _exit(0);
    }

    printf("Sending SIGCONT to stopped job PID: %d...\n", pid);
    kill(pid, SIGCONT);

    printf("Updating Job Table status: [1] RUNNING in background.\n");
    printf("Shell prompt returned immediately: teja27@TEJA:~$ \n");

    waitpid(pid, NULL, 0);
    printf("Background job PID %d completed successfully.\n", pid);
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week11_task4.c
teja27@TEJA:~$ gcc week11_task4.c -o week11_task4
teja27@TEJA:~$ ./week11_task4
[Task 4] Resume Stopped Jobs in Background (bg %1)
Sending SIGCONT to stopped job PID: 893...
Updating Job Table status: [1] RUNNING in background.
Shell prompt returned immediately: teja27@TEJA:~$
Background job PID 893 completed successfully.
teja27@TEJA:~$ █
```